

LittleBit-2: Maximizing the Spectral Energy Gain in Sub-1-Bit LLMs via Latent Geometry Alignment

Banseok Lee¹ Youngmin Kim¹

Abstract

We identify the Spectral Energy Gain in extreme model compression, where low-rank binary approximations outperform tiny-rank floating-point baselines for heavy-tailed spectra. However, prior attempts fail to realize this potential, trailing state-of-the-art 1-bit methods. We attribute this degradation to Latent Geometry Misalignment: standard singular vectors exhibit high coherence (spiky distribution), the worst-case geometry for binary quantization. To realize this gain, we propose **LittleBit-2**, a framework employing Internal Latent Rotation and Joint Iterative Quantization (Joint-ITQ). This approach acts as a geometric preconditioner, aligning coherent latent distributions with the binary hypercube with zero inference overhead. Empirically, LittleBit-2 establishes a new state-of-the-art in the sub-1-bit regime (1~0.1 bpp) on Llama-2 and Llama-3, matching the fidelity of leading 1-bit baselines.

1. Introduction

Scaling Large Language Models (LLMs) drives remarkable performance gains (Brown et al., 2020; Touvron et al., 2023) but faces a rigid hardware constraint: the memory wall (Gholami et al., 2022). A 70B model requires ~140 GB of VRAM in FP16, prohibiting consumer-grade deployment. Consequently, model compression (Han et al., 2015) has shifted from an optimization technique to a deployment necessity. While Post-Training Quantization (PTQ) has standardized 4-bit precision (Frantar et al., 2022; Lin et al., 2024), recent works have pushed towards the 1-bit frontier (e.g., BitNet (Wang et al., 2023), OneBit (Xu et al., 2024)), demonstrating that LLMs can retain capabilities with ternary or binary weights.

¹Samsung Research, Seoul, Korea. Correspondence to: Youngmin Kim <ym1012.kim@samsung.com>.

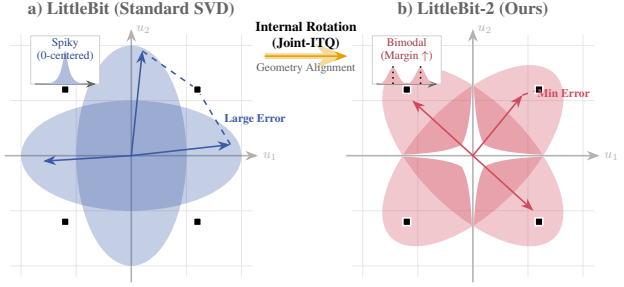


Figure 1. **Latent Geometry Alignment.** (a) Standard singular vectors exhibit high coherence (spiky distribution), clustering along the axes. This creates a geometric mismatch with the binary quantization targets (black dots at $(\pm 1, \pm 1)$). (b) **LittleBit-2** employs Internal Latent Rotation via Joint-ITQ. Acting as a geometric preconditioner, it rotates latent factors to align with binary hypercube diagonals. This minimizes quantization noise (min error) and maximizes the optimization margin (Bimodal distribution).

However, a critical efficiency gap remains. Even at 1-bit, a 70B model demands ~15 GB of memory, which is still prohibitive for edge devices. This necessitates *sub-1-bit* compression. **LittleBit** (Lee et al., 2025) approached this via a **Low-Rank Binary** architecture. We validate their design through Spectral Theory (Eckart & Young, 1936), demonstrating that low-rank binary approximations outperform tiny-rank floating-point counterparts for heavy-tailed weights. Despite this theoretical potential, LittleBit trails the state-of-the-art 1-bit baseline (Xu et al., 2024). We attribute their underperformance to Latent Geometry Misalignment.

To realize this gain, we propose **LittleBit-2**, a framework employing Internal Latent Rotation and Joint Iterative Quantization (Joint-ITQ). As illustrated in Figure 1, this acts as a geometric preconditioner, aligning coherent latent distributions with the binary hypercube.

Empirically, LittleBit-2 establishes a new state-of-the-art in the sub-1-bit regime (1~0.1 bpp) on Llama-2 and Llama-3, matching the fidelity of leading 1-bit baselines.

Our contributions are threefold:

- **Theoretical Diagnosis:** We identify the theoretical superiority of Low-Rank Binary approximation over Tiny-Rank FP16 baselines in heavy-tailed spectra. We formulate this via the Spectral Break-Even Condition,

proving that rank expansion outweighs quantization noise.

- **Geometric Alignment via Joint-ITQ:** We propose LittleBit-2, which employs Joint-ITQ to rotate latent factors. This aligns the latent distribution with the binary hypercube vertices, maximizing the decision margin and minimizing quantization error.
- **SOTA Sub-1-bit Performance:** LittleBit-2 establishes a new state-of-the-art in the sub-1-bit regime (down to ~ 0.1 bpp) on Llama-2 and Llama-3, matching the fidelity of leading 1-bit baselines.

2. Related Work

2.1. Extreme Quantization

Recent approaches to 1-bit LLMs, such as BitNet b1.58 (Wang et al., 2023; Ma et al., 2024) introduced ternary weights $\{-1, 0, +1\}$, demonstrating that LLMs can be trained from scratch with extreme quantization. However, these require prohibitive training cost compared to compressing pre-trained weights. Alternatively, several PTQ frameworks (e.g., BiLLM (Huang et al., 2024), ARB-LLM (Li et al., 2024), STBLLM (Dong et al., 2024)) achieve nominal sub-1-bit precision by retaining salient weights in high precision while binarizing the rest. However, these methods rely on auxiliary binary masks and sparse indices to locate outliers. In practice, this metadata introduces additional memory overhead.

1-bit and sub-1-bit QAT presents unique challenges. Unlike moderate compression where PTQ suffices, this regime hits an irreducible error floor due to the collapse of representational capacity. Consequently, **Quantization-Aware Knowledge Distillation (QAKD)** (Kim et al., 2019; Liu et al., 2024a; Du et al., 2024) has emerged as a highly effective strategy to recover fidelity. Existing methods like OneBit (Xu et al., 2024) and LittleBit (Lee et al., 2025) utilize this paradigm to map weights to extremely low-bit domains. **Building on this protocol, our work specifically tackles the initialization bottleneck via geometric alignment.**

2.2. Incoherence Processing

To mitigate quantization errors caused by weight outliers, Incoherence Processing via orthogonal rotation has emerged as a critical preconditioner. Existing methods like QuIP# (Tseng et al., 2024), QuaRot (Ashkboos et al., 2024), and SpinQuant (Liu et al., 2024b) apply global rotations, specifically learned or randomized Hadamard transforms, to full weight and activation matrices. While this effectively delocalizes outliers into quantization-friendly Gaussian distributions, it often incurs inference overhead due to the requisite

online Hadamard transforms. In contrast, LittleBit-2 applies rotation strictly to the latent factors ($\hat{U}, \hat{V}$) during the weight decomposition process. This approach isolates geometric alignment to the initialization phase, effectively resolving the optimization instability **without incurring any additional inference overhead compared to LittleBit.**

2.3. Low-Rank & Structured Compression

Beyond pure quantization, low-rank decomposition forms a parallel direction for compression. LoSparse (Li et al., 2023) combines low-rank approximation with column-wise sparsity to capture coherent and outlier patterns jointly, and Prune-and-Factorize (Ren & Zhu, 2024) couples first-order pruning with factorization. A separate line allocates the rank budget non-uniformly across layers, via Bayesian optimization (Ji et al., 2024) or mixed discrete-continuous rank learning (Idelbayev & Carreira-Perpinán, 2020), while Zhang & Saab (2025) establish data-driven recovery guarantees for low-rank compression in general. These directions can be combined with LittleBit-2 (for example, through spectral-decay-aware rank allocation on top of our geometric alignment), but this work focuses on what makes sub-1-bit quantization effective and on the corresponding initialization methodology.

3. Background

3.1. The LittleBit Architecture

Conventional quantization approximates the weight matrix $W \in \mathbb{R}^{d_{out} \times d_{in}}$, facing a lower bound of 1 bit per parameter (bpp) even with binary weights. To achieve sub-1-bit compression, LittleBit (Lee et al., 2025) utilizes a Latent Factorization Quantization framework. It decouples the parameter count from matrix dimensions via low-rank factorization (U_r, V_r), enabling fractional effective bit-rates (e.g., < 1 bpp) even with binary weights.

Tri-Scale Latent Factorization. The framework decomposes W into binary latent factors $U_b \in \{\pm 1\}^{d_{out} \times r}$ and $V_b \in \{\pm 1\}^{d_{in} \times r}$ with rank $r \ll \min(d_{in}, d_{out})$. To explicitly decouple magnitude from sign (i.e., $U \approx |U| \odot U_b$), LittleBit approximates the magnitude envelopes $|U|$ and $|V|$ via Rank-1 factorization. Consequently, LittleBit employs a symmetric **Scale-Binary-Scale-Binary-Scale** architecture to recover magnitude precision. It defines three learnable FP16 scaling components: row scale $h \in \mathbb{R}^{d_{out}}$, a central latent scale vector $l \in \mathbb{R}^r$, and column scale $g \in \mathbb{R}^{d_{in}}$, formulated as:

$$\hat{W} = \underbrace{\text{diag}(h)}_{\text{Scale}} \cdot \underbrace{U_b}_{\text{Binary}} \cdot \underbrace{\text{diag}(l)}_{\text{Scale}} \cdot \underbrace{V_b^T}_{\text{Binary}} \cdot \underbrace{\text{diag}(g)}_{\text{Scale}} \quad (1)$$

This structure ensures that the binary latent factors are sandwiched by floating-point scales, allowing for extreme com-

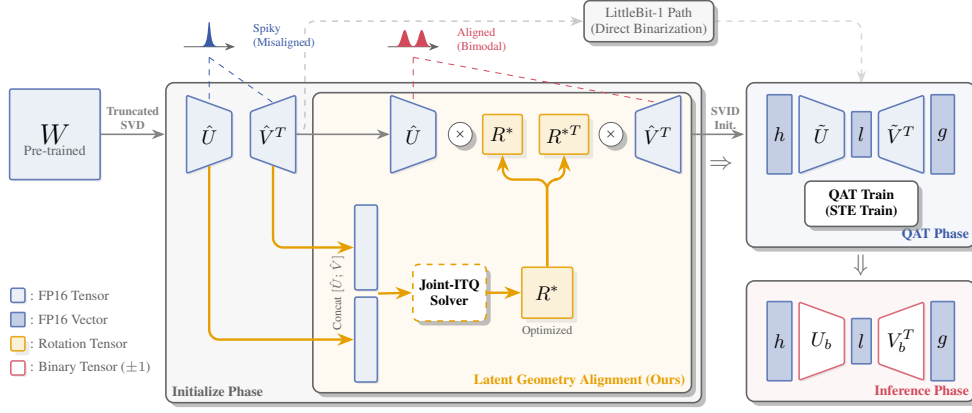


Figure 2. **The LittleBit-2 Framework Pipeline.** Starting from a truncated SVD of the pretrained weight W , **LittleBit-2** (lower path) explicitly addresses the geometric misalignment. The factors $\tilde{U}$ and $\tilde{V}$ are concatenated and fed into the **Joint-ITQ** solver to optimize an orthogonal rotation R . This rotation is applied ($\times$) to $\tilde{U}$ and $\tilde{V}$, transforming the spiky latent distribution (blue histogram) into an aligned bimodal distribution (red histogram). Finally, Dual-SVID and QAT extract the FP16 scales (h, l, g) and learn the binary factors (U_b, V_b^T) .

pression ratios by controlling the rank r while preserving the magnitude dynamics of the original weights. Following LittleBit, we employ this in a residual structure repeated twice; see Appendix G for details.

Initialization. Training latent factors $(\hat{U}, \hat{V})$ from scratch is unstable due to the sign function’s gradient mismatch. LittleBit mitigates this via a Dual-SVID strategy, initializing latent factors from singular vectors (binarized as $U_b = \text{sign}(U)$ during the forward pass (Bengio et al., 2013)). To derive the FP16 scales (h, l, g) , it performs Rank-1 approximation on the absolute values of these vectors (i.e., $\text{SVD}_1(|\hat{U}|)$) to explicitly decouple the magnitude envelope from the sign (Appendix C). **In this work, we do not propose a new architecture but refine the initialization to stabilize training and boost performance, ensuring robust optimization.**

3.2. Latent Geometry Misalignment

While LittleBit achieves sub-1-bit compression, a performance gap remains compared to state-of-the-art 1-bit models. For instance, under similar memory budgets, **OneBit** (Xu et al., 2024) achieves a perplexity of 8.36 on Llama-2-7B, whereas LittleBit lags at 9.08.

We attribute this performance degradation to a **Latent Geometry Misalignment**. The latent factors derived from standard SVD frequently exhibit high coherence, inheriting the spiky nature of the weight distribution (Tseng et al., 2024; Liu et al., 2024b) in a small subset of dominant channels. In these dimensions, information concentrates in a few outliers while the majority of values remain near zero. This represents a worst-case scenario for binary quantization ($\text{sign}(x)$), as binarization destroys the latent structure, causing high quantization noise. **LittleBit-2 aims to resolve this by geometrically realigning the latent factors before**

binarization to maximize information retention.

4. The LittleBit-2 Framework

In this section, we formulate the sub-1-bit quantization as a constrained spectral optimization problem. We derive the Spectral Break-Even Condition, establishing the condition where the information gain from rank expansion outweighs the quantization noise. We then propose **LittleBit-2**, which satisfies this condition via Latent Geometry Alignment, solving a joint orthogonal Procrustes problem to maximize the geometric margin of quantized factors.

4.1. Why Low-Rank Binary?

Problem Formulation We formulate sub-1-bit quantization as a constrained optimization problem to justify Low-Rank Binary architectures (Lee et al., 2025). Given a pre-trained weight $W \in \mathbb{R}^{d_{out} \times d_{in}}$ with rank $d = \min(d_{in}, d_{out})$, we seek an approximation $\hat{W}$ minimizing reconstruction error $\| \cdot \|_F^2$ ($:= \mathcal{E}$) under a strict bit-budget $\mathcal{B}$:

$$\min_{\hat{W}} \|W - \hat{W}\|_F^2 \quad \text{s.t.} \quad \text{Bits}(\hat{W}) \leq \mathcal{B} \quad (2)$$

We analyze two distinct strategies under the budget $\mathcal{B}$:

- **Strategy A (Tiny-Rank FP16):** Retains minimal rank r_A . Error is dominated by truncation: $\mathcal{E}_A \approx \mathcal{E}_{\text{trunc}}(r_A)$.
- **Strategy B (Low-Rank Binary):** Retains expanded rank $r_B \approx 16r_A$ with 1-bit quantization. The error comprises reduced truncation and quantization noise: $\mathcal{E}_B \approx \mathcal{E}_{\text{trunc}}(r_B) + \mathcal{E}_{\text{quant}}(r_B)$, aligning with LittleBit.

Spectral Analysis We analyze the trade-off under a fixed budget $\mathcal{B}$, modeling singular values with a power-law decay $\sigma_k \approx Ck^{-\gamma}$ characteristic of LLM weights (Martin &

Mahoney, 2021). Following their definition, we classify the spectrum as heavy-tailed if $\gamma \leq 0.5$ and light-tailed if $\gamma > 0.5$. For analytical tractability, we approximate the discrete spectrum via continuous integration.

Proposition 4.1 (Spectral Break-Even Condition). *Let the quantization noise be $\mathcal{E}_{\text{quant}}(r) = \int_0^r \Lambda \sigma(x)^2 dx$ and truncation error be $\mathcal{E}_{\text{trunc}}(r) = \int_r^\infty \sigma(x)^2 dx$, assuming an average distortion coefficient Λ . Strategy B (binary, rank r_B) outperforms Strategy A (FP16, rank $r_A \ll r_B$) if and only if the **Tail Energy Gain** outweighs the **Quantization Cost**:*

$$\underbrace{\int_{r_A}^{r_B} \sigma(x)^2 dx}_{\text{Tail Gain}} > \underbrace{\int_0^{r_B} \Lambda \sigma(x)^2 dx}_{\text{Quantization Cost}} \quad (3)$$

This implies a critical threshold γ^* ; Strategy B is superior for heavy-tailed distributions ($\gamma < \gamma^*$).

Proposition 4.1 establishes a critical spectral decay rate γ^* such that for any distribution with $\gamma < \gamma^*$, the Low-Rank Binary strategy is theoretically superior to the Tiny-Rank FP16 approximation. This trade-off is visualized in Figure 9, provided in Appendix B. Empirically, Llama-2 7B exhibits a break-even point at $\gamma^* \approx 0.36$ (Section 5.1). 73% of linear layers exhibit $\gamma < 0.36$, confirming that the heavy-tailed structure of modern LLM weights intrinsically favors the Low-Rank Binary architecture over FP16 truncation.

As γ is intrinsic to pre-trained weights, the distortion coefficient Λ remains the only controllable variable. Minimizing Λ shifts the break-even threshold γ^* higher, extending applicability to lighter-tailed models, and widens the margin between Tail Gain and Quantization Cost (Eq. 3), reducing approximation error.

4.2. The Distortion Coefficient Λ

While treated as a constant in Proposition 4.1, Λ is practically governed by latent factor geometry. With γ fixed, we focus on minimizing Λ by analyzing the relationship between vector geometry and quantization noise.

Consider a latent vector $\mathbf{u} \in \mathbb{R}^r$ (a row of latent factor $\hat{U}$). In our low-rank binary architecture, $\mathbf{u}$ is approximated by a binary vector $\mathbf{b} = \text{sign}(\mathbf{u})$ and an optimal scalar scale $h_i \in \mathbb{R}$. The quantization error is defined as $\mathcal{E}(\mathbf{u}) = \min_{\alpha} \|\mathbf{u} - \alpha \mathbf{b}\|_2^2$.¹

We define the **Local Distortion Coefficient** as the ratio of the error energy to the signal energy: $\lambda(\mathbf{u}) := \mathcal{E}(\mathbf{u}) / \|\mathbf{u}\|_2^2$.

Lemma 4.2 (Distortion-Geometry Duality). *The optimal local distortion coefficient is strictly determined by the ratio*

¹While LittleBit employs Rank-1 approximation ($|U| \approx h U_u^T$) in practice, we analyze the channel-wise scale case here for analytical tractability. This abstraction is valid because scalar distortion serves as a theoretical lower bound for Rank-1 error; the geometric dependency remains the fundamental determinant.

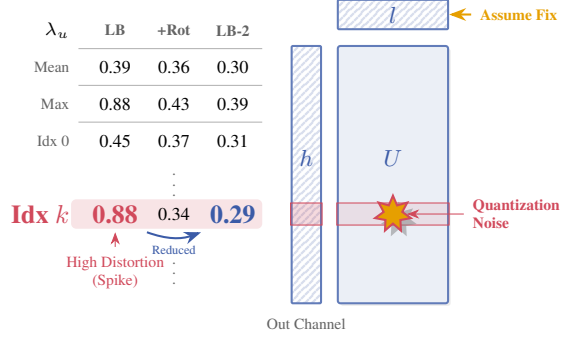


Figure 3. Latent Geometry Misalignment. We visualize the local distortion coefficient λ across latent rows. Standard initialization (LB) suffers from geometric outliers (max $\lambda \approx 0.88$), which act as spikes that degrade the precision of the shared floating-point scales. LittleBit-2 effectively suppresses these outliers through internal rotation, reducing the peak distortion to 0.29 and minimizing quantization noise.

of the squared L_1 norm to the squared L_2 norm of the vector:

$$\lambda(\mathbf{u}) = 1 - \frac{1}{r} \left(\frac{\|\mathbf{u}\|_1}{\|\mathbf{u}\|_2} \right)^2 \quad (4)$$

Equation 4 (proven in Appendix A.1) shows that noise minimization is equivalent to maximizing the vector denseness ($\|\mathbf{u}\|_1 / \|\mathbf{u}\|_2$). This ratio is inversely related to the Coordinate Incoherence defined in the next section, serving as the bridge between geometry and quantization fidelity. Since the weight is reconstructed via $\hat{W} \approx \hat{U} \hat{V}^T$, assuming statistical independence between the quantization errors of the factors, the global distortion Λ compounds the local distortions of both interacting factors:

$$\Lambda_{i,j} \approx 1 - (1 - \lambda_u(\mathbf{u}_i))(1 - \lambda_v(\mathbf{v}_j)) \quad (5)$$

Since minimizing the global distortion Λ essentially reduces to suppressing the local distortion λ of the factors, we focus our subsequent analysis on the local geometry defined by λ .

Eq. 3 requires minimizing λ . However, standard SVD typically yields latent factors with near-worst-case λ , exhibiting high coherence in dominant channels. This concentration creates a high-distortion barrier, violating the inequality in Eq. 3.

Definition 4.3 (Coordinate Incoherence). Let $U \in \mathbb{R}^{d_{out} \times r}$ be an orthogonal matrix. We define the coordinate incoherence $\mu(U)$ based on the element-wise infinity norm:

$$\mu(U) = \sqrt{d} \max_{i,j} |U_{ij}| \quad (6)$$

High μ implies energy concentration (spiky), whereas low μ indicates uniform information spread.

To quantify the impact of coherence on quantization, we contrast two geometric extremes derived from Lemma 4.2. The distortion λ is strictly governed by the vector’s density,

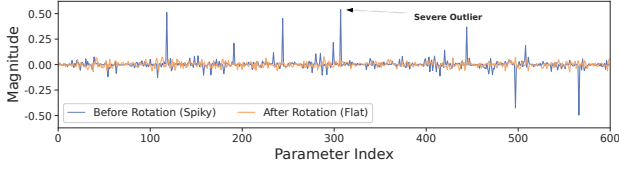


Figure 4. Visualization of Latent Geometry Alignment. Histograms of latent factor $\hat{U}$ derived from the $q\text{-proj}$ (W_Q) in the middle (15th) layer of Llama-2 7B. Applying Internal Rotation transforms the distribution into a Gaussian (Orange), effectively mitigating outliers.

which links directly to the Coordinate Coherence $\mu(U)$. In the worst-case scenario (High Coherence), latent vectors align with coordinate axes (e.g., $\mathbf{u} \approx [1, 0, \dots, 0]$). In this sparse regime, the L_1 norm collapses to the L_2 norm, pushing the distortion to its theoretical maximum $\lambda_{\text{worst}} \approx 1.0$, where binary quantization fails completely. Conversely, the best-case scenario (Low Coherence) occurs when the vector energy is democratized across all dimensions (dense). Here, the ratio $\|\mathbf{u}\|_1 / \|\mathbf{u}\|_2$ is maximized, driving the distortion λ toward zero. As illustrated in Figure 3, standard initialization suffers from these high-distortion spikes, whereas our geometric alignment successfully suppresses them.

Analyzing the latent factors of Llama-2 7B (15th layer $q\text{-proj}$, Figure 4), we observe that standard singular vectors exhibit a highly skewed distribution (Kurtosis ≈ 16.8) with heavy tails reaching maximum distortions of $\lambda_{\text{max}} \approx 0.88$. This empirical proximity to the worst-case barrier demonstrates that standard singular vectors inherently possess a geometry hostile to binary quantization.

4.3. Latent Geometry Preconditioning

To mitigate *High-Distortion Barrier*, we propose Internal Latent Rotation, applying a randomized orthogonal matrix $R \in \mathbb{R}^{r \times r}$ to the latent factors. This exploits the rotational invariance of the factorization:

$$W \approx \hat{U}\hat{V}^T = \hat{U}(RR^T)\hat{V}^T = (\hat{U}R)(\hat{V}R)^T = \tilde{U}\tilde{V}^T \quad (7)$$

where $\tilde{U} = \hat{U}R$ and $\tilde{V} = \hat{V}R$ denote the rotated latent factors. Since R is orthogonal ($RR^T = I$), the reconstruction remains exact in floating-point precision, but the coordinate distribution of the factors is fundamentally altered to be quantization-friendly.

Theorem 4.4 (Delocalization via Rotation). *Let R be a random orthogonal matrix. By Levy’s Lemma (Concentration of Measure) (Ledoux, 2001), the mass of the rotated vectors $\tilde{U}$ concentrates around the expectation, forming a Gaussian-like distribution. This transformation maximizes the element-wise density, driving the expected distortion down to the **Gaussian Limit** (proof in Appendix A.2):*

$$\mathbb{E}[\lambda_{\text{Rot}}] \approx 1 - \frac{1}{r} \left(\sqrt{\frac{2r}{\pi}} \right)^2 = 1 - \frac{2}{\pi} \approx 0.36 \quad (8)$$

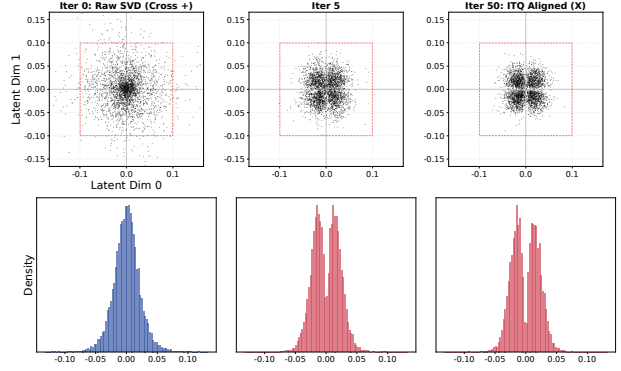


Figure 5. Evolution of Latent Geometry via Joint-ITQ. Histograms of latent factors ($\hat{U}, \hat{V}$) from the Llama-2 7B 15th layer K projection (first two latent dimensions). (Left) Raw SVD factors exhibit high coherence, concentrating probability mass near the zero decision boundary while containing significant outliers. (Right) Joint-ITQ (Iter 50) transforms this into a **bimodal distribution**, explicitly aligning the geometry with the binary vertices $\{\pm 1\}$ to maximize the decision margin.

This rotation provides Coarse Alignment, stabilizing the Rank-1 scale approximation ($|U| \approx h \cdot l_u^T$). In standard SVD, high coherence forces shared scaling factors to accommodate outliers, overestimating the dynamic range for the majority of inliers. By enforcing an isotropic Gaussian distribution, we ensure that the magnitude matrix $|\tilde{U}|$ becomes nearly uniform. This homogenization allows the Rank-1 outer product to approximate the underlying geometry with reduced residual error, satisfying the structural constraints of the architecture.

Empirically, on the representative Llama-2 weight analyzed in Section 4.3, this rotation reduces the mean distortion (λ) to 0.36, matching the theoretical limit, and suppresses the maximum distortion from 0.88 to 0.43. This confirms that scaling factors are no longer dominated by outliers.

4.4. Joint-Iterative Latent Quantization

While Random Rotation mitigates outliers by enforcing isotropy, it introduces rotational ambiguity. The resulting vectors often reside in the uncertainty zone near zero, misaligned with quantization targets. To minimize λ ($\lambda \rightarrow 0$), we break this isotropy by formulating the alignment as a Joint Orthogonal Procrustes Problem (Gong et al., 2012). We seek a rotation R^* minimizing the distance between the joint latent manifold $Z = [\hat{U}; \hat{V}]$ and binary vertices. We optimize a **shared** rotation R over the concatenated manifold to align **both** latent factors simultaneously. As R is orthogonal, this maintains reconstruction consistency $\tilde{W} \approx (\hat{U}R)(\hat{V}R)^T = \hat{U}\hat{V}^T$ while minimizing quantization error for both factors.

$$Z = \begin{bmatrix} \hat{U} \\ \hat{V} \end{bmatrix} \quad (9)$$

Algorithm 1 LittleBit-2 Initialization via Latent Geometric Alignment

```

1: Input: Weight  $W$ , Target Rank  $r$ , Iterations  $T$ 
2: Output: Binary latent factors  $\tilde{U}, \tilde{V}$ , Scales  $h, g, l$ 
3:  $U, \Sigma, V^T \leftarrow \text{SVD}(W)$ 
4:  $\hat{U} \leftarrow U_{:,1:r} \Sigma_{1:r}^{1/2}, \quad \hat{V} \leftarrow V_{:,1:r} \Sigma_{1:r}^{1/2}$ 
5:  $Z \leftarrow \text{Concat}([\hat{U}; \hat{V}]) \triangleright Z \in \mathbb{R}^{(d_{in}+d_{out}) \times r}$ 
6: Initialize  $R \in \mathbb{R}^{r \times r}$  as random orthogonal matrix.
7: for  $t = 1$  to  $T$  do
8:    $B \leftarrow \text{sign}(ZR) \triangleright \text{Project to Binary Vertices}$ 
9:    $\Phi, \Omega, \Psi^T \leftarrow \text{SVD}(B^T Z) \triangleright \text{Solve Procrustes Prob.}$ 
10:   $R \leftarrow \Psi \Phi^T \triangleright \text{Update Rotation}$ 
11: end for
12:  $\tilde{U} \leftarrow \tilde{U} R, \quad \tilde{V} \leftarrow \tilde{V} R \triangleright \text{Rotate factors}$ 
13: // Dual-SVID Init. (Scale extraction via Rank-1 SVD)
14:  $u_{\text{vec}}, u_{\text{rk}} \leftarrow \text{SVD}_1(|\tilde{U}|), \quad v_{\text{vec}}, v_{\text{rk}} \leftarrow \text{SVD}_1(|\tilde{V}|)$ 
15:  $h \leftarrow u_{\text{vec}}, \quad l \leftarrow u_{\text{rk}} \odot v_{\text{rk}}, \quad g \leftarrow v_{\text{vec}} \triangleright \text{Set scales}$ 
16: return  $\tilde{U}, \tilde{V}, h, g, l$ 
    
```

Our objective is to find the rotation R that best aligns this joint manifold with the vertices of the binary hypercube B . This is formalized as the following minimization problem:

$$\min_{R, B} \|B - ZR\|_F^2 \quad \text{s.t.} \quad R^T R = I_r, \quad B \in \{\pm 1\}^{(d_{in}+d_{out}) \times r} \quad (10)$$

We solve this non-convex objective via alternating minimization, iterating between the binary code update ($B = \text{sign}(ZR)$) and the rotation update via SVD on $B^T Z$ (Algorithm 1).

This optimization transforms the unimodal Gaussian into a **Bimodal Distribution** aligned with the hypercube diagonals (Figure 5). By shifting mass from zero to $\{\pm 1\}$, it maximizes the Geometric Margin. On the representative Llama-2 weight analyzed in Section 4.3, this further suppresses the mean distortion coefficient to **0.30**, surpassing the theoretical Gaussian limit (≈ 0.36). The process incurs negligible overhead, converging within 50 iterations with few seconds (Appendix F.1).

5. Experiments

5.1. Synthetic Validation

To validate Proposition 4.1, we conducted reconstruction experiments on synthetic weights. We generated random matrices $W \in \mathbb{R}^{4096 \times 4096}$ with singular values decaying according to a power-law $\sigma_k \propto k^{-\gamma}$, varying the decay rate $\gamma \in [0.1, 0.8]$. We compared the reconstruction fidelity of **LittleBit-2** against baselines under identical memory constraints. The approximation employs initialized 1-bit quantization to measure the reconstruction error, whereas Tiny-Rank FP16 is obtained via truncated SVD.

Figure 6 (Top) illustrates the reconstruction error (MSE) as a function of γ . We observe a distinct phase transition governed by the spectral shape. While standard LittleBit out-

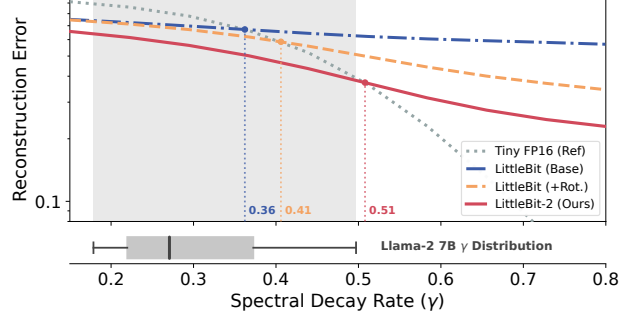


Figure 6. Spectral Break-Even Analysis. (Top) Reconstruction error (MSE) vs. spectral decay rate γ . LittleBit-2 (with Joint-ITQ) extends the range where 1-bit quantization outperforms FP16. (Bottom) Distribution of γ in Llama-2 7B weights. The shaded region (5th–95th percentile) shows real-world weights fall mostly within the heavy-tailed regime. LittleBit-2 dominates this region, whereas the baseline fails in the upper quantile.

performs Tiny-Rank FP16 only in the heavy-tailed regime ($\gamma \lesssim 0.36$), our geometric interventions progressively extend this superiority. Specifically, applying internal rotation pushes the break-even point to $\gamma \approx 0.41$, and **LittleBit-2** (with Joint-ITQ) further extends it to $\gamma \approx 0.51$. This confirms that for heavy-tailed spectra, the information gain from massive rank expansion ($r_{\text{bin}} \approx 16r_{\text{fp}}$) outweighs the precision loss from 1-bit quantization.

To confirm that this expanded effective range covers the operational regime of modern LLMs, Figure 6 (Bottom) compares the empirical decay rates from **all linear layers of Llama-2 7B** against our method’s performance. Here, all gammas are calculated by log linear regression of real weights. The majority (90%) of observed rates lie within $\gamma \in [0.19, 0.47]$ (median: 0.27), a range that completely overlaps with the 1-bit LittleBit-2’s optimal reconstruction zone. This confirms that the heavy-tailed spectral properties of LLMs are intrinsically suited for sub-1-bit compression.

5.2. Main Results

Experimental Setup. We evaluate our method on Llama-2 7B, 13B (Touvron et al., 2023) and Llama-3 8B (Grattafiori et al., 2024). For evaluation metrics, we measure the perplexity (PPL) on WikiText-2 (Merity et al., 2016) using a sequence length of 2048. To assess reasoning capabilities, we report the average zero-shot accuracy across five benchmark tasks: HellaSwag (Zellers et al., 2019), ARC-Easy, ARC-Challenge (Clark et al., 2018), PIQA (Bisk et al., 2020), and Winogrande (Sakaguchi et al., 2021).

Baselines. We compare LittleBit-2 against a wide range of quantization methods. For **GPTQ** (Frantar et al., 2022), **EfficientQAT** (Chen et al., 2025), **STBLLM** (Dong et al., 2024), **BiLLM** (Huang et al., 2024), and **ARB-LLM** (Li et al., 2024) we utilized their officially released codes to

Table 1. Main Results on Llama-2 and Llama-3. Perplexity (PPL) on **WikiText-2** and average accuracy across **5 zero-shot tasks**. We denote **Body** (blocks only) and **Total** (including head and embedding) memory footprints in GB. At 0.1 bpp, LittleBit-2 compresses body weights to $<1\%$, leaving the footprint dominated by the fixed LM Head with Embedding. See Appendix D for detailed results.

METHOD	BITS	LLAMA-3 8B					LLAMA-2 7B					LLAMA-2 13B				
		PPL↓	AVG↑	MEM (GB)			PPL↓	AVG↑	MEM (GB)			PPL↓	AVG↑	MEM (GB)		
				BODY (%)	TOTAL (%)				BODY (%)	TOTAL (%)				BODY (%)	TOTAL (%)	
FP16	16	6.10	72.97	14.0 (100)	16.1 (100)		5.47	68.88	13.0 (100)	13.5 (100)		4.88	71.87	25.4 (100)	26.1 (100)	
GPTQ	2	1480	36.01	2.0 (14.2)	4.1 (25.4)		52.2	38.79	1.8 (14.2)	2.4 (17.5)		23.6	44.34	3.6 (14.1)	4.2 (16.3)	
EFFICIENTQAT	2	9.80	63.49	2.0 (14.2)	4.1 (25.4)		7.17	62.93	1.8 (14.2)	2.4 (17.5)		6.08	68.06	3.6 (14.1)	4.2 (16.3)	
<i>1-bit Regime</i>																
BiLLM	1.1	59.37	37.73	2.5 (18.2)	4.6 (28.9)		29.00	40.81	2.4 (18.2)	2.9 (21.4)		21.53	49.12	4.6 (18.1)	5.3 (20.2)	
ARB-LLM	1.1	27.63	52.75	2.4 (17.5)	4.5 (28.2)		15.88	52.32	2.3 (17.5)	2.8 (20.7)		12.13	57.98	4.5 (17.8)	5.1 (19.5)	
ONEBIT	1.0	13.09	52.23	0.9 (6.4)	3.0 (18.6)		8.36	54.72	0.8 (6.4)	1.4 (10.0)		7.41	58.47	1.6 (6.4)	2.3 (8.7)	
LITTLEBIT	1.0	16.30	47.11	0.9 (6.3)	3.0 (18.6)		9.08	52.81	0.8 (6.3)	1.3 (10.0)		8.18	51.49	1.6 (6.3)	2.3 (8.7)	
LITTLEBIT-2 (OURS)	1.0	11.53	57.33	0.9 (6.3)	3.0 (18.6)		8.27	55.79	0.8 (6.3)	1.3 (10.0)		7.37	60.53	1.6 (6.3)	2.3 (8.7)	
<i>Sub-1-bit Regime (Extreme Compression)</i>																
STBLLM	0.55	241.96	36.76	-	-		32.93	43.40	-	-		27.05	45.62	-	-	
LITTLEBIT	0.55	18.47	44.59	0.5 (3.5)	2.6 (16.1)		10.47	48.49	0.5 (3.6)	1.0 (7.3)		9.24	51.76	0.9 (3.5)	1.6 (5.9)	
LITTLEBIT-2 (OURS)	0.55	14.01	51.33	0.5 (3.5)	2.6 (16.1)		9.65	50.97	0.5 (3.6)	1.0 (7.3)		8.56	55.64	0.9 (3.5)	1.6 (5.9)	
LITTLEBIT	0.1	26.11	41.80	0.1 (0.7)	2.2 (13.7)		15.92	41.33	0.1 (0.7)	0.6 (4.6)		15.09	40.99	0.2 (0.7)	0.8 (3.2)	
LITTLEBIT-2 (OURS)	0.1	23.74	41.64	0.1 (0.7)	2.2 (13.7)		14.70	42.06	0.1 (0.7)	0.6 (4.6)		13.94	42.18	0.2 (0.7)	0.8 (3.2)	

reproduce the results. Note that all these baselines utilize a group size of 128. To ensure a rigorous comparison with **LittleBit** (Lee et al., 2025) and **OneBit** (Xu et al., 2024), we adopted the identical training configuration (including the number of tokens, epochs, and learning rate) as detailed in (Lee et al., 2025).

Performance Analysis. Table 1 summarizes the performance comparison. LittleBit-2 establishes a new state-of-the-art in the low-bit quantization regime. In the 1-bit setting, LittleBit-2 significantly outperforms binary quantization methods such as BiLLM and ARB-LLM. LittleBit-2 achieves a perplexity of **11.53** on Llama-3 8B, significantly surpassing the baseline LittleBit (16.30). Our method matches or surpasses the performance of OneBit (11.53 vs. 13.09 in PPL, 57.33% vs. 52.23% in Avg).

We observe a scaling anomaly in the baseline LittleBit, where Llama-2 13B underperforms Llama-2 7B on zero-shot tasks (51.49% vs. 52.81%). This suggests that standard initialization struggles to handle the increased optimization difficulty of larger models. LittleBit-2 successfully resolves this issue, restoring the expected performance scaling (60.53% vs. 55.79%). This confirms that latent geometry alignment provides the necessary stability for training larger-scale models.

The superiority of LittleBit-2 becomes most pronounced in the deep compression regime. As the bit-width decreases to 0.55 bpp, STBLLM exhibits severe performance collapse (PPL 241.96 on Llama-3). While the baseline LittleBit maintains functionality, it still suffers from optimization instability (PPL 18.47). In contrast, LittleBit-2 retains high fidelity (PPL 14.01), reducing the perplexity gap with the FP16 model significantly.

Even in the extreme 0.1 bpp regime, compressing the

model body to $<1\%$ of its original size (~ 0.1 GB for 8B models), LittleBit-2 remains functional. It achieves a perplexity of 23.74 on Llama-3 8B, outperforming the baseline LittleBit (PPL 26.11). This result aligns with the theoretical prediction in Section 4. LittleBit-2 effectively retains the heavy-tailed information essential for minimal functionality.

Scalability on Large-Scale Models To verify scalability on larger, recent architectures, we extended evaluation to **Gemma-3 27B**. We focused on the extreme compression regime (0.1 bpp) to test the limits of latent information retention.

As summarized in Table 2, **LittleBit-2** demonstrates robust scalability even in the extreme compression regime. Notably, in the 0.1 bpp setting, while the Tiny-Rank FP approximation collapses (PPL > 35), LittleBit-2 maintains functional capabilities with a PPL of 16.38 and an average zero-shot accuracy of 47.06%, significantly outperforming the baseline.

Table 2. Results on Gemma-3 27B. Comparison of Perplexity (PPL) and average zero-shot accuracy (Avg). The evaluation focuses on the extreme 0.1-bit regimes. LittleBit-2 consistently outperforms the baselines, showing gains in the 0.1 bpp setting.

METHOD	BITS	PPL↓	AVG (%)↑
FP16 (Ref)	16.0	4.58	79.48
FP (TINY-RANK)	0.1	37.53	39.15
LITTLEBIT	0.1	17.72	45.52
LITTLEBIT-2 (OURS)	0.1	16.38	47.06

5.3. Ablation Study

We conduct a component-wise ablation study on Llama-3 8B across standard (1.0 bpp) and extreme (0.1 bpp) regimes to isolate geometric alignment contributions. Table 3 dis-

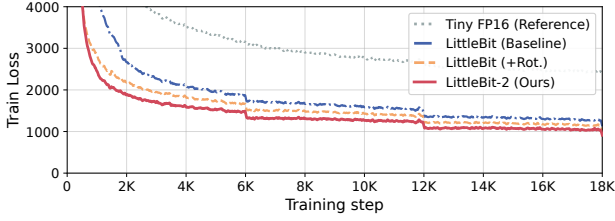


Figure 7. Training Convergence. Comparison of training loss trajectories for 1-bit QAT on Llama-2 7B. **Tiny-Rank FP16** (Gray) suffers from high error. While standard **LittleBit** (Blue) captures more capacity, it exhibits slower convergence due to latent misalignment. Applying **Internal Rotation** (Orange) improves optimization speed by mitigating outliers, and **LittleBit-2** (Red, with Joint-ITQ) achieves the fastest convergence and lowest final loss.

sects the performance gains. First, the collapse of Tiny-Rank FP (PPL 59.44) at 0.1 bpp contrasts with the stability of LittleBit Base (PPL 26.11). This empirically validates the **Spectral Break-Even Condition** (Proposition 4.1), confirming that in the heavy-tailed spectral regime, the information gain from massive rank expansion significantly outweighs the precision loss incurred by binarization. Second, geometric alignment yields progressive improvements. While Random Rotation reduces error by mitigating outliers (16.30 $\rightarrow$ 12.63), **LittleBit-2** (Joint-ITQ) achieves the best performance (11.53) by explicitly aligning latent factors with the binary hypercube vertices, thereby maximizing the geometric decision margin.

Table 3. Ablation Study on Llama-3 8B. Comparison of WikiText-2 Perplexity (PPL). The results highlight the failure of Tiny-Rank FP16 in extreme regimes (0.1 bpp) and the progressive improvement from geometric alignment (Rotation $\rightarrow$ ITQ).

METHOD	0.1 BPP	1.0 BPP
Original Model (FP16)	6.10	
FP (TINY-RANK)	59.44	26.24
LITTLEBIT (BASE)	26.11	16.30
+ RANDOM ROTATION	24.16	12.63
LITTLEBIT-2 (OURS)	23.74	11.53

6. Discussion

6.1. Training Stability

We analyze the training dynamics to understand why geometric alignment leads to superior performance. Figure 7 illustrates the training loss trajectories of 1-bit quantization on Llama-2 7B. The **Tiny-Rank FP16** baseline saturates early at a high loss plateau, confirming the fundamental bottleneck of minimal rank. In contrast, **LittleBit-2** achieves the lowest final loss and accelerates convergence in the early training phase compared to the baseline.

To investigate this acceleration, we analyze the Sign Flipping Ratio, defined as the percentage of binary parameters changing state (± 1) per step. As shown in Figure 8, the

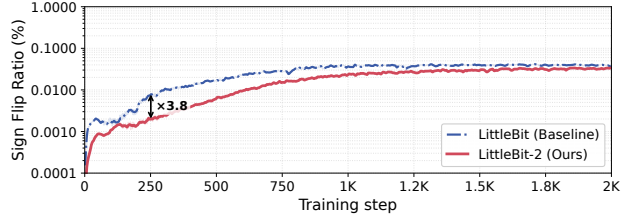


Figure 8. Sign Flipping Analysis. The percentage of binary parameters changing signs per step during the first 2k steps (7% of total) under the same setting as Figure 7. LittleBit-2 (Red) significantly reduces oscillation compared to the baseline (Blue). This confirms that in the early stage of training geometric margin stabilizes the weights against stochastic gradient noise.

standard LittleBit baseline exhibits a persistently high flipping rate. This indicates that early in training, many latent parameters are trapped near the decision boundary ($x = 0$), oscillating due to gradient noise rather than learning features.

We attribute the stability of LittleBit-2 to the Geometric Margin induced by Joint-ITQ. While standard random rotation yields a unimodal Gaussian distribution concentrated at zero (maximizing instability), Joint-ITQ enforces a bimodal separation aligned with the binary vertices. By pushing latent factors away from the unstable zero-crossing region, our method effectively locks the weights against high-frequency gradient noise. This geometric preconditioning stabilizes the optimization landscape, enabling effective feature learning from the initial iterations.

6.2. Inference Efficiency

While Latent Geometry Alignment improves quantization accuracy, it does not compromise inference efficiency. LittleBit-2 retains the *identical* inference architecture as the original LittleBit framework (Lee et al., 2025). Consequently, our method directly inherits the established computational advantages of the Low-Rank Binary structure.

The core advantage of this architecture lies in its MatMul-free design. By decomposing weights into low-rank binary factors, it replaces FP16 GEMV operations with Bitwise Operations (BOPs), while reducing the total parameter count. For instance, a Llama-2 7B MLP layer at 0.3 bpp reduces computational complexity from 90.2M FLOPs (add+mul) to 13M FLOPs (add) with 13M sign flips.

Empirical benchmarks reported in prior work (Lee et al., 2025) further validate this efficiency on hardware. At the kernel level, a Llama-2 70B MLP layer compressed to 0.1 bpp demonstrates an **11.6 $\times$ speedup** (0.288 ms $\rightarrow$ 0.025 ms) compared to the cuBLAS FP16 baseline. This efficiency translates to substantial gains in end-to-end generation; a 0.1 bpp Llama-2 7B model achieves a **2.46 $\times$ speedup** (82.6 tokens/s $\rightarrow$ 203.2 tokens/s) in a 128-token decoding, batch

1 setting. Since LittleBit-2 shares this underlying computational graph, it guarantees the same level of acceleration.

7. Conclusion

We present **LittleBit-2**, a framework that realizes the spectral gain in sub-1-bit LLMs by resolving Latent Geometry Misalignment via Joint-ITQ. By transforming coherent singular vectors into quantization-friendly bimodal distributions, our approach maximizes the geometric decision margin and realizes the information-theoretic advantage of the Low-Rank Binary strategy. LittleBit-2 achieves state-of-the-art performance down to 0.1 bpp, effectively matching 1-bit baselines and confirming that geometrically aligned extreme compression is a viable path for deploying foundation models on memory-constrained edge devices. We provide a detailed discussion of limitations in Appendix I.

Future directions include developing adaptive rank allocation guided by spectral decay (γ), exploring hybrid architectures combining FP components with LittleBit, extending Latent Geometry Alignment to non-language architectures such as Vision Transformers and multimodal models, and employing advanced optimization strategies to further accelerate training.

Impact Statement

We present a method for extreme model compression that significantly lowers the computational and energy costs of deploying LLMs. This work has positive societal impacts by promoting energy-efficient computing and democratizing access to state-of-the-art AI technology. While deploying powerful models on edge devices introduces challenges regarding the control of potential misuse, it also offers significant benefits for data privacy.

References

- Ashkboos, S., Mohtashami, A., Croci, M. L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoefler, T., and Hensman, J. Quarot: Outlier-free 4-bit inference in rotated llms. *Advances in Neural Information Processing Systems*, 37:100213–100240, 2024.
- Bengio, Y., Léonard, N., and Courville, A. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- Bisk, Y., Zellers, R., Gao, J., Choi, Y., et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 7432–7439, 2020.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Chen, M., Shao, W., Xu, P., Wang, J., Gao, P., Zhang, K., and Luo, P. Efficientqat: Efficient quantization-aware training for large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10081–10100, 2025.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Dong, P., Li, L., Zhong, Y., Du, D., Fan, R., Chen, Y., Tang, Z., Wang, Q., Xue, W., Guo, Y., et al. Stblm: Breaking the 1-bit barrier with structured binary llms. *arXiv preprint arXiv:2408.01803*, 2024.
- Du, D., Zhang, Y., Cao, S., Guo, J., Cao, T., Chu, X., and Xu, N. Bitdistiller: Unleashing the potential of sub-4-bit llms via self-distillation. *arXiv preprint arXiv:2402.10631*, 2024.
- Eckart, C. and Young, G. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.
- Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M. W., and Keutzer, K. A survey of quantization methods for efficient neural network inference. In *Low-power computer vision*, pp. 291–326. Chapman and Hall/CRC, 2022.
- Gong, Y., Lazebnik, S., Gordo, A., and Perronnin, F. Iterative quantization: A procrustean approach to learning binary codes for large-scale image retrieval. *IEEE transactions on pattern analysis and machine intelligence*, 35(12):2916–2929, 2012.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Han, S., Mao, H., and Dally, W. J. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015.

- Huang, W., Liu, Y., Qin, H., Li, Y., Zhang, S., Liu, X., Magno, M., and Qi, X. Billm: Pushing the limit of post-training quantization for llms. *arXiv preprint arXiv:2402.04291*, 2024.
- Idelbayev, Y. and Carreira-Perpinán, M. A. Low-rank compression of neural nets: Learning the rank of each layer. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 8049–8059, 2020.
- Ji, Y., Xiang, Y., Li, J., Xia, Q., Ye, Z., Duan, X., Wang, Z., Chen, K., and Zhang, M. Adaptive feature-based low-rank compression of large language models via bayesian optimization. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 4152–4168, 2024.
- Kim, J., Bhargat, Y., Lee, J., Patel, C., and Kwak, N. Qkd: Quantization-aware knowledge distillation. *arXiv preprint arXiv:1911.12491*, 2019.
- Ledoux, M. *The concentration of measure phenomenon*. Number 89. American Mathematical Soc., 2001.
- Lee, B., Kim, D., You, Y., and Kim, Y. Littlebit: Ultra low-bit quantization via latent factorization. *arXiv preprint arXiv:2506.13771*, 2025.
- Li, Y., Yu, Y., Zhang, Q., Liang, C., He, P., Chen, W., and Zhao, T. Lospase: Structured compression of large language models based on low-rank and sparse approximation. In *International Conference on Machine Learning*, pp. 20336–20350. PMLR, 2023.
- Li, Z., Yan, X., Zhang, T., Qin, H., Xie, D., Tian, J., Kong, L., Zhang, Y., Yang, X., et al. Arb-llm: Alternating refined binarizations for large language models. *arXiv preprint arXiv:2410.03129*, 2024.
- Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of machine learning and systems*, 6:87–100, 2024.
- Liu, Z., Oguz, B., Zhao, C., Chang, E., Stock, P., Mehdad, Y., Shi, Y., Krishnamoorthi, R., and Chandra, V. Llm-qat: Data-free quantization aware training for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 467–484, 2024a.
- Liu, Z., Zhao, C., Fedorov, I., Soran, B., Choudhary, D., Krishnamoorthi, R., Chandra, V., Tian, Y., and Blankevoort, T. Spinqant: Llm quantization with learned rotations. *arXiv preprint arXiv:2405.16406*, 2024b.
- Ma, S., Wang, H., Ma, L., Wang, L., Wang, W., Huang, S., Dong, L., Wang, R., Xue, J., and Wei, F. The era of 1-bit llms: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024.
- Martin, C. H. and Mahoney, M. W. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for learning. *Journal of Machine Learning Research*, 22(165):1–73, 2021.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.
- Ren, S. and Zhu, K. Low-rank prune-and-factorize for language model compression. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pp. 10822–10832, 2024.
- Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Tseng, A., Chee, J., Sun, Q., Kuleshov, V., and De Sa, C. Quip#: Even better llm quantization with hadamard incoherence and lattice codebooks. *arXiv preprint arXiv:2402.04396*, 2024.
- Wang, H., Ma, S., Dong, L., Huang, S., Wang, H., Ma, L., Yang, F., Wang, R., Wu, Y., and Wei, F. Bitnet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023.
- Xu, Y., Han, X., Yang, Z., Wang, S., Zhu, Q., Liu, Z., Liu, W., and Che, W. Onebit: Towards extremely low-bit large language models. *Advances in Neural Information Processing Systems*, 37:66357–66382, 2024.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.
- Zhang, S. and Saab, R. Theoretical guarantees for low-rank compression of deep neural networks. *Applied and Computational Harmonic Analysis*, pp. 101837, 2025.

A. Detailed Theoretical Analysis

A.1. Proof of Lemma 4.2 (Distortion-Geometry Duality)

Let $\mathbf{b} = \text{sign}(\mathbf{u}) \in \{-1, +1\}^r$. The objective is to minimize $f(\alpha) = \|\mathbf{u} - \alpha\mathbf{b}\|_2^2$. Differentiating with respect to α :

$$\frac{\partial f}{\partial \alpha} = -2\mathbf{b}^T(\mathbf{u} - \alpha\mathbf{b}) = -2(\mathbf{b}^T\mathbf{u} - \alpha\mathbf{b}^T\mathbf{b}) = 0 \quad (11)$$

Solving for α , and noting that $\mathbf{b}^T\mathbf{b} = r$ and $\mathbf{b}^T\mathbf{u} = \sum |u_i| = \|\mathbf{u}\|_1$:

$$\alpha^* = \frac{\mathbf{b}^T\mathbf{u}}{\mathbf{b}^T\mathbf{b}} = \frac{\|\mathbf{u}\|_1}{r} \quad (12)$$

Substituting α^* back into the normalized error function $\lambda(\mathbf{u}) = \mathcal{E}(\mathbf{u})/\|\mathbf{u}\|_2^2$:

$$\mathcal{E}(\mathbf{u}) = \|\mathbf{u}\|_2^2 - 2\alpha^*(\mathbf{b}^T\mathbf{u}) + (\alpha^*)^2(\mathbf{b}^T\mathbf{b}) = \|\mathbf{u}\|_2^2 - \frac{\|\mathbf{u}\|_1^2}{r} \quad (13)$$

$$\therefore \lambda(\mathbf{u}) = 1 - \frac{1}{r} \left(\frac{\|\mathbf{u}\|_1}{\|\mathbf{u}\|_2} \right)^2 \quad \blacksquare \quad (14)$$

A.2. Proof of Theorem 4.4 (Delocalization via Rotation)

Part 1: Random Rotation (Gaussian Limit). Let $\tilde{\mathbf{u}} = \mathbf{u}R$ where R is a random orthogonal matrix. By the concentration of measure, elements of $\tilde{\mathbf{u}}$ converge to a Gaussian distribution $\tilde{u}_i \sim \mathcal{N}(0, \sigma^2)$ with $\sigma^2 = \|\mathbf{u}\|_2^2/r$. For a Gaussian variable x , $\mathbb{E}[|x|] = \sigma\sqrt{2/\pi}$. Thus, the expected L_1 norm is:

$$\mathbb{E}[\|\tilde{\mathbf{u}}\|_1] \approx r \cdot \sigma \sqrt{\frac{2}{\pi}} = \sqrt{\frac{2r}{\pi}} \|\mathbf{u}\|_2 \quad (15)$$

Substituting this into Lemma 4.2:

$$\mathbb{E}[\lambda_{\text{Rot}}] \approx 1 - \frac{1}{r} \left(\sqrt{\frac{2r}{\pi}} \right)^2 = 1 - \frac{2}{\pi} \approx 0.3634 \quad (16)$$

This proves that Random Rotation strictly bounds the distortion, reducing the error by $\sim 63.7\%$ compared to the worst-case SVD ($\lambda \approx 1$).

Part 2: Joint-ITQ (Optimal Alignment). The Joint-ITQ algorithm minimizes the objective $\mathcal{J}(R) = \|\text{sign}(ZR) - ZR\|_F^2$. Minimizing this distance is equivalent to maximizing the trace:

$$\max_R \text{Tr}(\text{sign}(ZR)^T(ZR)) = \max_R \sum_{i,j} \text{sign}((ZR)_{ij})(ZR)_{ij} = \max_R \sum_{i,j} |(ZR)_{ij}| = \max_R \|ZR\|_1 \quad (17)$$

Thus, the ITQ optimization explicitly searches for a rotation R that maximizes the L_1 norm of the latent factors. From Lemma 4.2, λ is monotonically decreasing with respect to the L_1 norm. Since ITQ starts from a random rotation and iteratively increases the L_1 norm, it guarantees a distortion coefficient strictly lower than Random Rotation:

$$\lambda_{\text{ITQ}} \leq \lambda_{\text{Rot}} < \lambda_{\text{SVD}} \quad (18)$$

In the limit of perfect alignment (where vectors align with hypercube vertices), $\|ZR\|_1 = \sqrt{r}\|ZR\|_2$, yielding $\lambda_{\text{ITQ}} = 0$. $\blacksquare$

B. Conceptual Visualization of the Spectral Gain

In this section, we provide a visual interpretation of the theoretical trade-off discussed in Section 4. Figure 9 illustrates the *Spectral Energy Gain* under a fixed memory budget.

For heavy-tailed distributions characteristic of LLMs (where $\gamma < \gamma^*$), **Strategy A (Tiny-Rank FP16)** suffers from severe truncation error, discarding significant information residing in the tail. In contrast, **Strategy B (Low-Rank Binary)** tolerates quantization noise to achieve massive rank expansion. As visually demonstrated, the tail gain (information recovered by higher rank) significantly outweighs the quantization cost (noise area), validating the superiority of the sub-1-bit architecture.

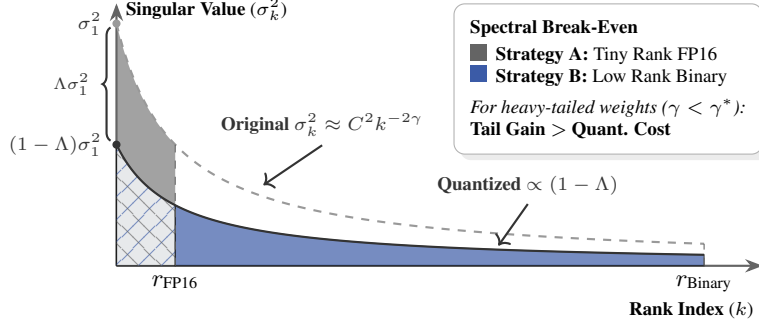


Figure 9. **Conceptual Illustration of the Spectral Gain.** Comparison between Strategy A (Tiny-Rank FP16) and Strategy B (Low-Rank Binary) under a fixed budget. While Strategy A suffers from rank starvation, Strategy B utilizes massive rank expansion to capture the heavy-tailed energy.

C. Detailed Architecture of LittleBit

In this section, we provide supplementary details on the initialization strategy and a comparative analysis of the LittleBit architecture, which were omitted from the main text.

C.1. Dual-SVID Initialization

Training a network with binary latent factors from scratch is notoriously unstable due to the non-differentiable nature of the sign function. Random initialization typically leads to training collapse. To address this, LittleBit employs **Dual-SVID (Sign-Value-Independent Decomposition)**, a specialized initialization scheme that decouples the initialization of binary geometry (Signs) and floating-point magnitudes (Values).

The Dual-SVID process operates in two main stages, assuming the weight matrix $\mathbf{W} \in \mathbb{R}^{d_{out} \times d_{in}}$:

1. **Subspace Approximation via SVD:** First, we approximate the pre-trained weight $\mathbf{W}$ using a standard singular value decomposition (SVD), truncated to the target rank r . We distribute the singular values equally to obtain the initialized latent factors $\hat{\mathbf{U}}$ and $\hat{\mathbf{V}}$ directly:

$$\mathbf{W} \approx \hat{\mathbf{U}} \hat{\mathbf{V}}^\top \quad \text{where} \quad \hat{\mathbf{U}} = U_r \Sigma_r^{1/2}, \quad \hat{\mathbf{V}} = V_r \Sigma_r^{1/2} \quad (19)$$

Here, $\hat{\mathbf{U}} \in \mathbb{R}^{d_{out} \times r}$ and $\hat{\mathbf{V}} \in \mathbb{R}^{d_{in} \times r}$. These factors preserve the optimal low-rank subspace direction and sign structure.

2. **Scale Extraction via Rank-1 Decomposition:** To initialize the floating-point scales $(\mathbf{h}, \mathbf{l}, \mathbf{g})$, we extract the magnitude envelopes from $\hat{\mathbf{U}}$ and $\hat{\mathbf{V}}$. Since the LittleBit architecture constraints the magnitude to a shared latent scale (Eq. 1), we employ a **Rank-1 approximation** on the absolute values of these factors:

$$|\hat{\mathbf{U}}| \approx \mathbf{h} \cdot \ell_u^\top, \quad |\hat{\mathbf{V}}| \approx \mathbf{g} \cdot \ell_v^\top \quad (20)$$

Here, $\mathbf{h} \in \mathbb{R}^{d_{out}}$ and $\mathbf{g} \in \mathbb{R}^{d_{in}}$ are directly assigned as the row and column scales, respectively. The vectors $\ell_u, \ell_v \in \mathbb{R}^r$ capture the relative importance of each latent dimension. The final central scale $\mathbf{l}$ is initialized as their element-wise product: $\mathbf{l} = \ell_u \odot \ell_v$.

This strategy effectively disentangles multi-dimensional magnitude variations, ensuring that the initial effective weight $\hat{\mathbf{W}}$ closely approximates the original $\mathbf{W}$ before quantization-aware training begins.

Algorithm 2 Dual-SVID Initialization for LittleBit

- 1: **Input:** Pre-trained weight W , Target rank r
- 2: **Step 1: SVD Decomposition**
- 3: Compute $W \approx U\Sigma V^T$ via randomized SVD.
- 4: Truncate to rank r to obtain U_r, Σ_r, V_r .
- 5: **Step 2: Latent Weight Initialization**
- 6: // Initialize full-precision latent weights for STE-based training
- 7: Assign truncated SVD components to latent weights: $\hat{U} \leftarrow U_r \Sigma_{1:r}^{1/2}, \hat{V} \leftarrow V_r \Sigma_{1:r}^{1/2}$.
- 8: **Step 3: Magnitude Initialization (Scale)**
- 9: Approximate magnitudes $|\hat{U}_r|$ and $|\hat{V}_r|$ using Rank-1 approximations to initialize row/col scales h, g, l .
- 10: **Output:** Latent weights $\hat{U}, \hat{V}$ and Scale factors h, g, l .

D. Extended Results

Table 4. Extended Results on Zero-shot Tasks. Detailed breakdown of zero-shot performance across Llama-2 and Llama-3 families. We report Perplexity (PPL) on WikiText-2 and accuracy for HellaSwag (Hella), WinoGrande (Wino), ARC-Easy (ArcE), ARC-Challenge (ArcC), and PIQA. Avg denotes the average accuracy of the five zero-shot tasks.

METHOD	BITS	LLAMA-3 8B								LLAMA-2 7B								LLAMA-2 13B							
		PPL↓	WINO↑	HELLA↑	ARCE↑	ARCC↑	PIQA↑	AVG↑		PPL↓	WINO↑	HELLA↑	ARCE↑	ARCC↑	PIQA↑	AVG↑		PPL↓	WINO↑	HELLA↑	ARCE↑	ARCC↑	PIQA↑	AVG↑	
FP16	16	6.10	73.24	79.09	80.01	53.16	79.33	72.97		5.47	68.59	75.93	76.18	46.08	77.64	68.88		4.88	72.30	79.36	79.50	48.98	79.22	71.87	
GPTQ	2	1480	49.33	27.36	27.40	22.87	53.08	36.01		52.2	49.80	34.20	29.55	24.57	55.82	38.79		23.6	50.59	44.03	40.11	25.85	61.10	44.34	
EFFICIENTQAT	2	9.80	65.67	67.78	68.98	39.93	75.08	63.49		7.17	64.17	67.52	71.04	38.05	73.89	62.93		6.08	69.22	74.43	75.04	44.80	76.82	68.06	
<i>1-bit Regime</i>																									
BILLM	1.1	59.37	49.27	30.44	32.11	20.99	55.82	37.73		29.00	51.14	33.76	36.57	23.12	59.47	40.81		21.53	57.22	44.71	51.81	27.39	64.47	49.12	
ARB-LLM	1.1	27.63	61.09	47.76	57.70	29.86	67.36	52.75		15.88	60.93	49.69	54.80	29.35	66.81	52.32		12.13	65.82	56.59	63.85	33.36	70.29	57.98	
ONEBIT	1.0	13.09	52.88	53.28	56.27	28.33	70.40	52.23		8.36	56.91	57.70	59.01	28.24	71.76	54.72		7.41	60.22	63.67	63.09	31.91	73.45	58.47	
LITTLEBIT	1.0	16.30	49.17	42.77	49.24	25.26	69.10	47.11		9.08	56.35	54.40	54.80	27.73	70.78	52.81		8.18	54.78	52.06	53.11	27.13	70.35	51.49	
LITTLEBIT-2	1.0	11.53	60.85	60.22	61.20	30.72	73.67	57.33		8.27	58.88	59.48	58.50	29.27	72.80	55.79		7.37	63.46	65.74	65.57	33.79	74.10	60.53	
<i>Sub-1-bit Regime</i>																									
STBLLM	0.55	241.96	48.15	28.71	29.59	23.12	54.24	36.76		32.93	52.80	37.77	40.78	24.49	61.15	43.40		27.05	57.38	38.75	45.50	24.49	61.97	45.62	
LITTLEBIT	0.55	18.47	51.30	36.30	46.76	22.70	65.89	44.59		10.47	52.96	46.20	49.54	25.09	68.66	48.49		9.24	55.64	53.18	53.28	26.96	69.75	51.76	
LITTLEBIT-2	0.55	14.01	55.96	50.71	54.21	26.02	69.75	51.33		9.65	55.49	51.05	52.19	27.22	68.88	50.97		8.56	59.43	57.35	59.68	29.61	72.14	55.64	
LITTLEBIT	0.1	26.11	51.70	30.02	40.32	23.46	63.49	41.80		15.92	51.22	30.55	39.02	24.40	61.48	41.33		15.09	51.14	31.18	38.13	21.93	62.57	40.99	
LITTLEBIT-2	0.1	23.74	50.67	30.66	41.04	22.53	63.28	41.64		14.70	52.57	32.61	39.73	22.27	63.11	42.06		13.94	50.36	33.24	40.87	23.72	62.73	42.18	

Table 5. Extended Results on Qwen3 Models. Comparison of Perplexity (PPL) and average zero-shot accuracy (Avg) on Qwen3 4B and 8B base models. The results demonstrate that LittleBit-2 consistently generalizes to the Qwen architecture, outperforming baselines in both standard (1.0 bpp) and extreme (0.1 bpp) compression regimes.

METHOD	BITS	QWEN3 4B		QWEN3 8B	
		PPL↓	AVG (%)↑	PPL↓	AVG (%)↑
FP16 (Ref)	16.0	7.90	70.55	7.00	73.66
LITTLEBIT (BASE)	1.0	14.92	46.88	13.60	50.11
LITTLEBIT-2 (OURS)	1.0	13.02	51.05	11.73	56.04

D.1. Aggregated Layer-Wise Distortion Statistics

To verify that latent geometry misalignment generalizes across LLM architectures rather than being a single-layer artifact, we aggregate the local distortion coefficient λ (Lemma 4.2) over all linear layers of Llama-3 8B (225 layers) and Qwen3 8B (253 layers), under three initializations: standard LittleBit (truncated SVD), random orthogonal rotation (LB+Rot.), and Joint-ITQ (LittleBit-2). Three findings emerge. First, under standard SVD, 0% of layers satisfy $\lambda_{\max} < 0.4$ on either model, indicating that high-distortion spikes are systematic rather than isolated outliers. Second, random rotation alone brings the mean λ below the threshold for 100% of layers but only partially suppresses the max (71–72%), motivating joint alignment of both factors. Third, Joint-ITQ achieves $\lambda < 0.4$ for 100% of layers on both the mean and max statistics across both models. Additionally, 83.6% of Llama-3 8B layers and 89.7% of Qwen3 8B layers exhibit heavy-tailed spectra ($\gamma < 0.4$), corroborating that the Spectral Break-Even Condition (Proposition 4.1) holds broadly across modern LLM architectures.

Table 6. **Aggregated layer-wise distortion statistics.** Mean local distortion coefficient λ and the fraction of layers satisfying $\lambda < 0.4$ across all linear layers of Llama-3 8B and Qwen3 8B. Standard SVD initialization leaves 0% of V -side factors below the $\lambda < 0.4$ threshold, confirming systematic misalignment. Joint-ITQ resolves this for 100% of layers on both factors across both models.

METHOD	LLAMA-3 8B (225 LAYERS)		QWEN3 8B (253 LAYERS)	
	$\lambda_{\text{mean}}(\text{avg.})$	$\lambda_{\text{mean}} < 0.4$	$\lambda_{\text{mean}}(\text{avg.})$	$\lambda_{\text{mean}} < 0.4$
LITTLEBIT (BASE)	0.390	73.8%	0.392	81.0%
LB + RANDOM ROT.	0.363	100.0%	0.363	100.0%
LITTLEBIT-2 (OURS)	0.243	100.0%	0.238	100.0%
	$\lambda_{\text{max}}(\text{avg.})$	$\lambda_{\text{max}} < 0.4$	$\lambda_{\text{max}}(\text{avg.})$	$\lambda_{\text{max}} < 0.4$
LITTLEBIT (BASE)	0.769	0.0%	0.778	0.0%
LB + RANDOM ROT.	0.394	72.0%	0.394	71.5%
LITTLEBIT-2 (OURS)	0.326	100.0%	0.330	100.0%

E. γ Validation

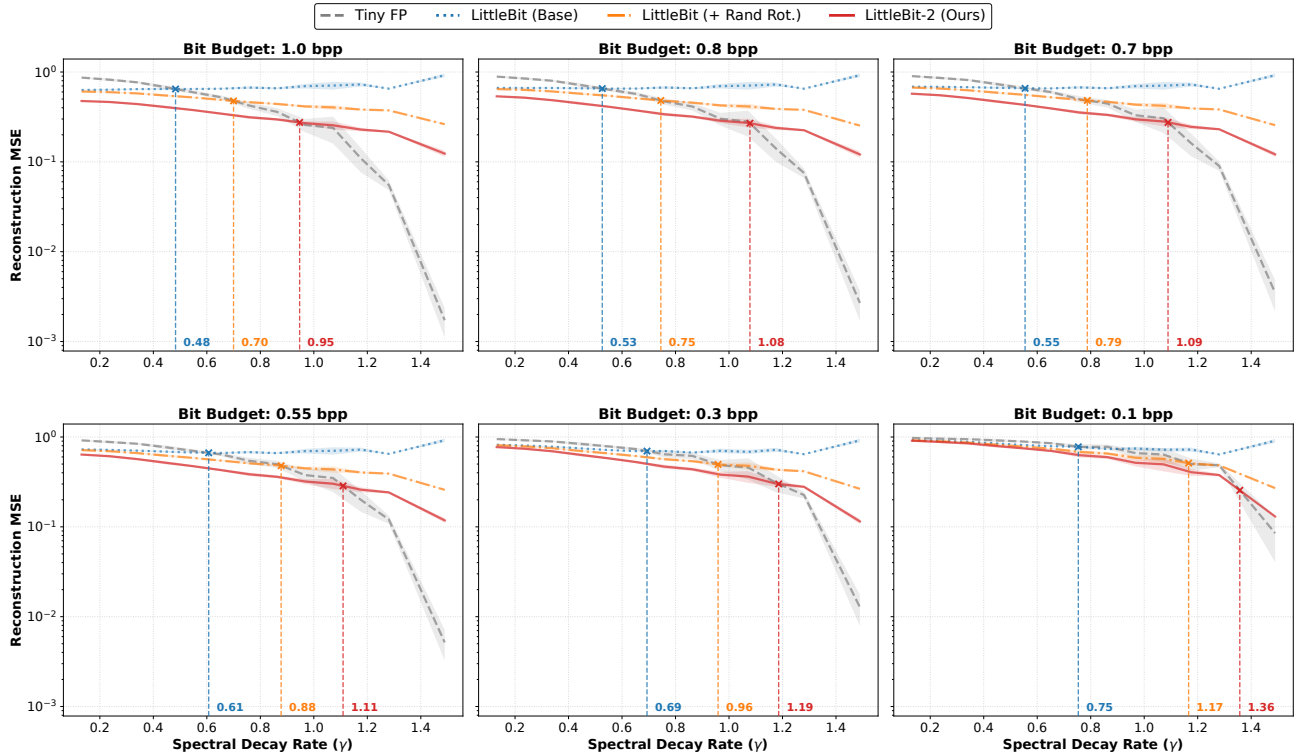


Figure 10. **Empirical Spectral Break-Even Analysis.** Reconstruction error (MSE) trends across varying γ values for bit-rates from 1.0 down to 0.1 bpp. The lines represent the average error calculated from the actual weights of 8 distinct LLMs. Consistent with the synthetic validation, **LittleBit-2** (Red) dominates the heavy-tailed regime ($\gamma \lesssim 0.5$) and significantly extends the superiority range over FP16 (Gray) compared to the baseline LittleBit (Blue), especially in extreme compression regimes.

To validate the theoretical bounds derived in Section 4.1 under realistic conditions, we performed a comprehensive trend analysis using actual weight matrices from 8 different models, including the Llama-2, Llama-3, and Gemma-3 families. We measured the initialization reconstruction error (MSE) across varying spectral decay rates (γ) for bit-budgets ranging from 1.0 down to 0.1 bpp. Specifically, we aggregated the MSE of all linear layers from the 8 models to observe the performance crossover points. As predicted, in the heavy-tailed regime (lower γ), the Tiny-Rank FP16 baseline suffers from rank starvation, whereas LittleBit-2 consistently achieves the lowest error. Notably, as the compression becomes more extreme (lower bpp), the crossing point shifts, indicating that our geometric alignment strategy becomes increasingly critical for preserving information in heavy-tailed distributions.

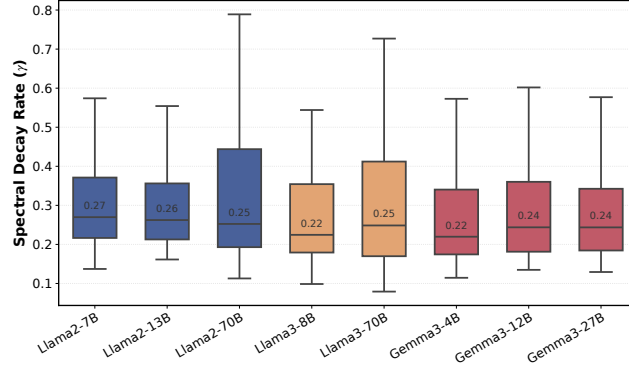


Figure 11. **Distribution of Spectral Decay Rates (γ) by Model.** Box plots summarizing the γ distribution of all linear layers across 8 different models. The median values consistently fall within the heavy-tailed range of $[0.26, 0.33]$.

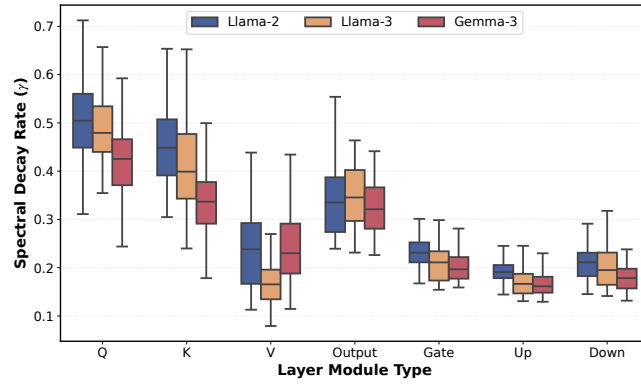


Figure 12. **Distribution of γ by Module Type.** Analysis of spectral decay rates grouped by projection types (Query, Key, Value, Output, etc.) across the Llama-2, Llama-3, and Gemma-3 families.

F. Extended Ablation Study

F.1. Sensitivity to Joint-ITQ Iterations

The **Joint-ITQ** algorithm iterates to optimally align latent factors with the binary hypercube. A critical hyperparameter is the number of iterations T , which governs the trade-off between the geometric alignment quality (reconstruction error) and the initialization overhead.

To determine the optimal T , we investigated the impact of iteration counts on both the Mean Squared Error (MSE) and the wall-clock initialization time. We performed this ablation on the query projection layer (`q_proj`) of the 15th block in Llama-2 7B, sweeping T from 0 to 100.

Convergence vs. Overhead. Figure 13 summarize the trajectory of error and time.

- **Reconstruction Fidelity (Red):** The MSE exhibits a sharp decay within the first 20 iterations and reaches a saturation point of diminishing returns near $T = 50$. This confirms that the joint optimization efficiently identifies the principal geometric alignment.
- **Computational Cost (Blue):** The initialization time increases linearly but remains computationally inexpensive. The baseline Random Rotation ($T = 0$) requires approximately 4 seconds due to SVD and allocation. Extending the optimization to $T = 50$ incurs a modest overhead of ~ 3 seconds (Total ~ 7 seconds).

Given that this process is a **one-time offline initialization** prior to QAT and incurs **zero inference overhead**, the additional latency is a highly cost-effective investment for the significant reduction in quantization noise. Consequently, we fix $T = 50$ for all main experiments to ensure robust convergence.

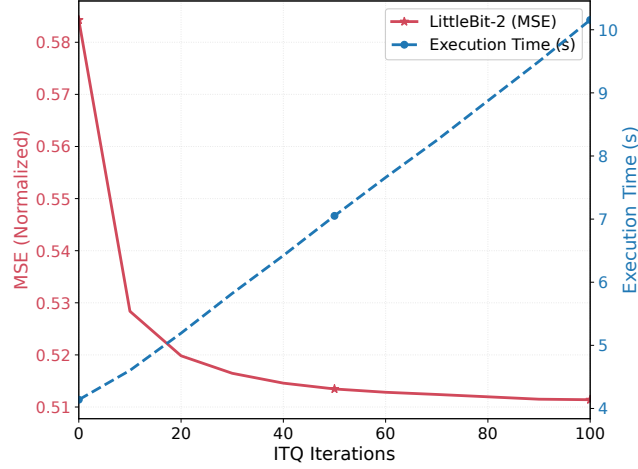


Figure 13. **Convergence vs. Overhead Analysis (Joint-ITQ).** Dual-axis plot showing the evolution of reconstruction error and initialization time. **Left Axis (Red):** The MSE exhibits a steep descent in the early phase and saturates around $T = 50$, indicating that the latent geometry rapidly converges to the optimal bimodal orientation. **Right Axis (Blue):** The initialization time scales linearly. The overhead for 50 iterations is marginal ($\sim 3s$) compared to the baseline ($T = 0$). We adopt $T = 50$ as the optimal operating point.

G. Efficacy of Residual

To maximize representational fidelity under a strict bit budget, we employ a residual architecture where the approximation is split into a primary path and a residual path ($W \approx \hat{W}_{\text{pri}} + \hat{W}_{\text{res}}$). In this section, we provide the theoretical justification for this design and empirically validate its impact.

A fundamental question arises: *Does splitting the rank into two paths offer a structural advantage over a single, wider path?* The answer depends on the precision domain.

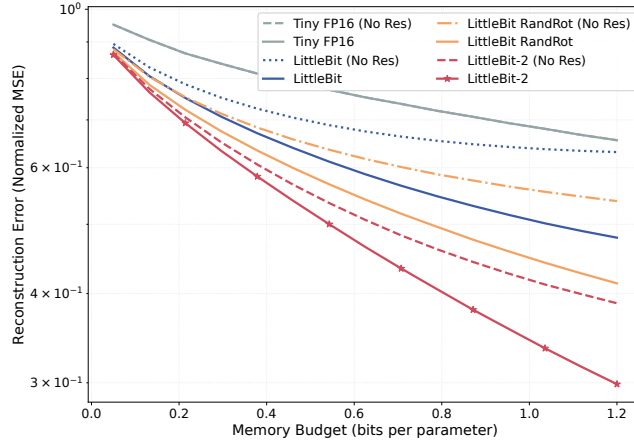


Figure 14. **Impact of Residual Architecture on Reconstruction Error.** Comparison of Mean Squared Error (MSE) versus Memory Budget. Solid lines indicate the residual architecture, and dashed lines indicate the single-path ablation. The results confirm a strict performance hierarchy: (1) **FP16 Invariance:** For Tiny-Rank FP16, the residual and non-residual lines overlap, confirming that splitting rank offers no benefit in the linear regime. (2) **Residual Gain:** For all binary methods (LittleBit, LittleBit(+Rot.), LittleBit-2, the residual architecture (Solid) consistently achieves lower MSE than the non-residual counterparts (Dashed). (3) **Geometric Dominance:** Notably, **LittleBit-2 (No Res)** outperforms the residual versions of prior methods (e.g., LittleBit(+Rot.)). This implies that efficacy of optimized geometric alignment (Joint-ITQ) is further maximized by the residual path.

Linear Regime (FP16). Due to the linearity of matrix factorization, decomposing a rank- $2r$ approximation into two rank- r matrices ($U_1 V_1^T + U_2 V_2^T$) is mathematically equivalent to a single rank- $2r$ SVD. Thus, for the **Tiny-Rank FP16** baseline, the residual topology offers no information-theoretic gain.

Non-Linear Regime (Binary). In the binary domain, the quantization operator $\mathcal{Q}(\cdot)$ introduces non-linear noise $E = W - \mathcal{Q}(W_{\text{pri}})$. A single binary path is strictly bound by this noise floor. However, in a residual architecture, the secondary path $\hat{W}_{\text{res}}$ explicitly targets the approximation of this error term E . This acts as a **Coarse-to-Fine Refinement** mechanism, where the primary path captures the structural skeleton and the residual path corrects the quantization noise.

We validate this hypothesis by analyzing the reconstruction error (MSE) across varying memory budgets (0.05 $\sim$ 1.2 bpp). Figure 14 compares the performance of single-path (“No Res”) and residual architectures across different methods.

As illustrated in Figure 14, the methods exhibit a clear hierarchy in reconstruction fidelity. The error decreases in the following order:

$$\text{FP16} \approx \text{FP16(NoRes)} > \text{LittleBit} > \text{RandRot} > \text{LittleBit-2(NoRes)} > \text{LittleBit-2}$$

This empirical evidence supports two key conclusions: First, residual connections are essential for binary quantization to mitigate noise. Second, the geometric alignment provided by LittleBit-2 acts as a fundamental performance multiplier, establishing a new state-of-the-art even without residual enhancement.

H. Memory Requirement Analysis

In this section, we explicitly define the memory requirement calculations used to evaluate the effective bits-per-parameter (bpp) and total model size. We denote the linear layer input dimension as d_{in} and output dimension as d_{out} . The total parameter count is $N = d_{\text{in}} \cdot d_{\text{out}}$. Unless otherwise stated, all high-precision scaling factors are calculated in FP16 (16-bit).

H.1. Standard Quantization Baselines

GPTQ / EfficientQAT. For 2-bit quantization methods utilizing group-wise scaling, such as GPTQ (Frantar et al., 2022) and EfficientQAT (Chen et al., 2025), we use a block size of $k = 128$. Each block requires one FP16 scale and one FP16 zero-point.

$$\mathcal{M}_{\text{GPTQ}} = \underbrace{2 \cdot N}_{\text{Quantized Weights}} + \underbrace{\frac{N}{128} \cdot (16 + 16)}_{\text{Scales \& Zero-points}} = 2.25 \cdot N \quad (\text{bits}) \quad (21)$$

OneBit. OneBit (Xu et al., 2024) decomposes the weight matrix into a 1-bit matrix and two high-precision scaling vectors (row and column scales).

$$\mathcal{M}_{\text{OneBit}} = \underbrace{1 \cdot N}_{\text{Binary Weights}} + \underbrace{16 \cdot (d_{\text{in}} + d_{\text{out}})}_{\text{Row \& Col Scales}} \quad (22)$$

H.2. Salient-Weight Binarization (BiLLM & ARB-LLM)

For BiLLM and ARB-LLM, we strictly follow the memory computation formulas presented in the ARB-LLM supplementary material (Li et al., 2024). These methods employ a block-wise structure ($k = 128$) and retain a subset of salient columns c in higher precision. To align with their notation, we map $n = d_{\text{out}}$ and $m = d_{\text{in}}$.

BiLLM. Based on Equations 134 and 135 in the ARB-LLM supplementary material, the memory footprint includes the salient weights (2-bit), the binary base (1-bit), two levels of block scaling factors, and metadata bitmaps.

$$\begin{aligned} \mathcal{M}_{\text{BiLLM}} = & \underbrace{2 \cdot n \cdot c + \left\lceil \frac{m}{k} \right\rceil \cdot 3n \cdot 16}_{\text{Second-order Binarization}} \\ & + \underbrace{n \cdot (m - c) + \left\lceil \frac{m}{k} \right\rceil \cdot 2n \cdot 16 \cdot 2}_{\text{First-order Binarization}} + \underbrace{n \cdot m + m}_{\text{Bitmaps}}, \end{aligned} \quad (23)$$

where c denotes the number of salient columns. In our analysis, we set $c = 128$.

ARB-LLM (RC). We utilize the ARB-RC (Row-Column) variant. Based on Equations 140 and 142 in the supplementary material, the memory is calculated as:

$$\begin{aligned} \mathcal{M}_{\text{ARB}} = & \underbrace{2 \cdot n \cdot c + \left(\left\lceil \frac{m}{k} \right\rceil \cdot 2n + 2c \right) \cdot 16}_{\text{Second-order Binarization}} \\ & + \underbrace{n \cdot (m - c) + \left(\left\lceil \frac{m}{k} \right\rceil \cdot n + (m - c) \right) \cdot 16 \cdot 2}_{\text{First-order Binarization}} + \underbrace{n \cdot m + m}_{\text{Bitmaps}} \end{aligned} \quad (24)$$

In our experiments, we set the number of salient columns c as 128.

H.3. LittleBit Framework (LittleBit-1 & LittleBit-2)

The memory requirements for LittleBit-1 and LittleBit-2 are identical, governed by the **Residual Low-Rank Binary Factorization**. We utilize a residual architecture ($path = 2$). The total memory bits $\mathcal{M}_{\text{LittleBit}}$ is calculated as:

$$\mathcal{M}_{\text{LittleBit}} = \underbrace{2r(d_{in} + d_{out} + 16)}_{\text{Rank-Dependent Terms}} + \underbrace{32(d_{in} + d_{out})}_{\text{Fixed I/O Scales}} \quad (25)$$

where r is the latent rank. The components are broken down as follows:

- **Binary Factors:** $2r(d_{in} + d_{out})$ bits. Two sets of binary matrices U_b, V_b for the main and residual paths.
- **I/O Scales:** $32(d_{in} + d_{out})$ bits. FP16 Input/Output vectors (g, h) for both paths ($2 \text{ paths} \times 16 \text{ bits} = 32$).
- **Latent Scales:** $32r$ bits. This term (derived from $2r \cdot 16$) represents the latent scaling overhead. In our optimized inference, the two scalar vectors (l_u, l_v) per path are merged into a single FP16 vector $l \in \mathbb{R}^r$ per path ($2 \text{ paths} \times r \times 16 \text{ bits} = 32r$).

This formula allows us to precisely determine the maximum rank r for any target bit-budget $\mathcal{B}$ by inverting the equation:

$$r = \left\lfloor \frac{\mathcal{B} \cdot N - 32(d_{in} + d_{out})}{2(d_{in} + d_{out} + 16)} \right\rfloor \quad (26)$$

Unlike the sparse baselines, LittleBit requires **no additional metadata indices or bitmaps**, ensuring that the calculated memory budget translates directly to the model size.

Model-Level Aggregation. To determine the total memory footprint reported in the experimental results (Section 5), we apply the aforementioned formulas to each linear layer within the model. We calculate the required bits based on the specific dimensions (d_{in}, d_{out}) of the Query, Key, Value, Output, Gate, Up, and Down projection layers. For all other parameters (e.g., LayerNorm, Embeddings), we assume standard FP16 precision. These per-layer memory costs are then aggregated across all transformer blocks defined by the model architecture. We report both the size of the model body (excluding the LM head) and the total model size (including the LM head) to provide a comprehensive comparison.

I. Limitations

Approximation Quality at Extreme Compression. While LittleBit-2 remains functional at 0.1 bpp, the quality degradation in this regime is substantial. The reported PPL of 23.74 on Llama-3 8B, despite being a notable improvement over baselines, still represents a significant gap to the FP16 reference (6.10). Closing this gap will likely require further architectural or algorithmic innovations, including more powerful QAT objectives.

Theoretical Relaxations. Proposition 4.1 relies on analytical simplifications including a uniform distortion coefficient Λ across all singular modes and independent quantization errors between factors. These assumptions facilitate closed-form spectral analysis but do not hold exactly in practice. Nevertheless, the empirical phase transitions observed on real LLM weights (Figure 6 and Appendix E) closely match the theoretical predictions, confirming the practical validity of these simplifications for guiding architectural design.

Compression-Speedup Gap. The theoretical compression ratio of LittleBit-2 substantially exceeds the realized inference speedup on current hardware. At 0.1 bpp, the storage reduction is $160\times$ relative to FP16, yet the kernel-level speedup is only $11.6\times$ (Section 6.2). This gap reflects current memory-bandwidth and bitwise-operation throughput limitations rather than algorithmic constraints. Specialized hardware support for low-bit bitwise GEMV operations would be required to close this gap.

Computational Cost of QAT. Although the Joint-ITQ initialization itself is inexpensive (~ 25 minutes for Llama-3 8B), the subsequent QAT phase requires substantial compute (~ 29 hours on $4\times H100$ for Llama-3 8B). This training overhead, common to QAKD-based extreme quantization approaches, limits accessibility for resource-constrained practitioners.

J. Generated Samples

We present qualitative generation samples from **Llama-2 13B** across different quantization regimes. To evaluate the preservation of knowledge, we use the fixed prompt: “*Turing is called the father of computer science. Computer science is*”.

J.1. 0.1-bit Regime (0.1 bpp)

LittleBit (0.1 bpp)

Output: Turing is called the father of computer science. Computer science is a branch of mathematics, and it’s the science that gives computers the ability to perform calculations on things they want. It doesn’t have much in common with logic or programming. But what about those who don’t know how to do math? That’s exactly what I’m doing here: I’ll teach you all these things you can remember from the ground up. You can even use them in your own programs! There are 24 different types of tutoring programs offered by The University of Michigan. There are online tutorials available for free (if you don’t already have one), there are

LittleBit-2 (Ours) (0.1 bpp)

Output: Turing is called the father of computer science. Computer science is a discipline that has been around for more than 10,000 years. Today it is one of the most important fields in the world and also one of the fastest growing areas of research. The term “Turing” refers to the theory of time which has been developed over many centuries by scientists from different parts of the globe who have applied this knowledge to create computers. The history of computers began with the invention of the steam engine in 1937 but after that, the technology became advanced enough so that the first person to use it was an engineer named John Wiley. He invented

J.2. 1-bit Regime (1.0 bpp)

LittleBit (1.0 bpp)

Output: Turing is called the father of computer science. Computer science is a sub-discipline within computer science and encompasses many fields, including artificial intelligence, software engineering, data structures and algorithms, computer architecture, algorithmics, computational complexity, and software development processes. The field also includes mathematical analysis and theoretical computer science. The first computer was developed by Alan Turing in 1946 and the world has never looked back since then. In his book “Alan Turing: A biography”, Sir John Simon wrote about Turing’s fascinating life and achievements. He had a very unique way of thinking and he was an extremely creative person who could

OneBit (Reference) (1.0 bpp)

Output: Turing is called the father of computer science. Computer science is a vastly important subject that many people are not very familiar with. The reason for this is because it is an extremely complicated field that has so much to offer in its various aspects. In fact, there are several other fields like artificial intelligence and machine learning as well. The most important thing about turing is that we can use it to build machines that actually do things similar to us humans. This is what makes it such an interesting concept. It is used to help solve some real-world problems that are difficult or impossible for human beings to figure out. This article will discuss how turing works, why it matters and

LittleBit-2 (Ours) (1.0 bpp)

Output: Turing is called the father of computer science. Computer science is a branch of science that studies the feasibility and potentiality of computers, their construction, operation, application and impact on society. In 1936 he proposed his theory of computation. In 1947 he developed the Turing test which was used to determine whether or not a machine could think for itself. It has since been widely adopted by researchers as an indicator of intelligence. He also contributed significantly to cryptography and computer security. Alan Turing was born in 1900 in London England. His family were Jews who had emigrated from Russia at the turn of the century.

K. Codes of Initialization Strategies

In this section, we provide the reference implementation for the three initialization strategies discussed in Section 4: (1) Standard LittleBit, (2) Random Rotation, and (3) LittleBit-2 with Joint-ITQ. We abstract the core logic into a `LatentInitializer` class.

K.1. Rank-1 Scale Extraction

We first define the helper function to approximate the floating-point scales (h, l, g) from the latent factor magnitudes.

```

1 def rank_one_decompose(X):
2     """
3     Approximates magnitude matrix X (|U| or |V|) via Rank-1 SVD.
4     Returns vectors u_vec, v_vec such that X ~ u_vec @ v_vec
5     """
6     # 1. SVD on magnitudes
7     U, S, Vh = torch.linalg.svd(X, full_matrices=False)
8
9     # 2. Distribute the dominant singular value
10    sqrt_s0 = torch.sqrt(S[0])
11    u_vec = (U[:, :1] * sqrt_s0).contiguous() # Shape: (N, 1)
12    v_vec = (sqrt_s0 * Vh[:1, :]).contiguous() # Shape: (1, M)
13
14    return u_vec, v_vec
    
```

Listing 1. Rank-1 Decomposition for Scale Extraction

K.2. Initialization Algorithms

The following methods implement the progression from the baseline to our proposed method.

```

1 class LatentInitializer:
2     def __init__(self, rank):
3         self.rank = rank
4
5     def _extract_scales(self, U, V):
6         """Extracts Tri-Scales (h, l, g) from latent factors."""
7         # Decompose magnitudes: |U| -> h, l_u AND |V| -> l_v, g
8         h, l_u = rank_one_decompose(torch.abs(U))
9         l_v, g = rank_one_decompose(torch.abs(V))
10
11        # Merge central scales
12        l = (l_u.view(-1) * l_v.view(-1)).view(1, self.rank)
13        return h, l, g
14
15    def init_standard(self, W):
16        """
17        LittleBit Initialization.
18        """
19        # 1. Truncated SVD
20        U_f, S_f, Vh_f = torch.linalg.svd(W.float(), full_matrices=False)
21
22        # 2. Symmetric Scaling
    
```

```

23     S_sqrt = torch.diag(torch.sqrt(S_f[:self.rank]))
24     U = U_f[:, :self.rank] @ S_sqrt
25     V = S_sqrt @ Vh_f[:self.rank, :]
26
27     # 3. Extract Scales
28     h, l, g = self._extract_scales(U, V)
29     return U, V, h, l, g
30
31 def init_random_rotation(self, W):
32     """
33     Internal Random Rotation.
34     """
35     # 1. Get Base Factors
36     U, V, _, _, _ = self.init_standard(W)
37
38     # 2. Apply Random Orthogonal Rotation
39     #  $W \sim (U @ R) @ (R.T @ V)$ 
40     R = torch.empty((self.rank, self.rank), device=W.device)
41     torch.nn.init.orthogonal_(R)
42
43     U_rot = U @ R
44     V_rot = R.t() @ V
45
46     # 3. Extract Scales from Rotated Factors
47     h, l, g = self._extract_scales(U_rot, V_rot)
48     return U_rot, V_rot, h, l, g
49
50 def init_joint_itq(self, W, n_iter=50):
51     """
52     LittleBit-2: Joint-ITQ Alignment.
53     """
54     # 1. Get Base Factors
55     U, V, _, _, _ = self.init_standard(W)
56
57     # 2. Construct Joint Manifold  $Z = [U; V.T]$ 
58     # Align input (V.T) and output (U) spaces simultaneously
59     Z = torch.cat([U, V.t()], dim=0)
60
61     # 3. Solve Joint Orthogonal Procrustes
62     R = torch.empty((self.rank, self.rank), device=W.device)
63     torch.nn.init.orthogonal_(R)
64
65     for _ in range(n_iter):
66         # Step A: Project to Binary Vertices
67         B = torch.sign(Z @ R)
68
69         # Step B: Optimal Rotation (SVD of  $B.T @ Z$ )
70         M = B.t() @ Z
71         Phi, _, Psi_t = torch.linalg.svd(M, full_matrices=False)
72         R = Psi_t.t() @ Phi.t() #  $R = V @ U.T$ 
73
74     # 4. Apply Aligned Rotation
75     U_aligned = U @ R
76     V_aligned = R.t() @ V
77
78     # 5. Extract Scales (Robust due to bimodal distribution)
79     h, l, g = self._extract_scales(U_aligned, V_aligned)
80     return U_aligned, V_aligned, h, l, g
    
```

Listing 2. Initialization Strategies: Baseline, Rotation, and Joint-ITQ

L. QAT Hyperparameters

For direct comparability, we follow the QAT configuration of LittleBit (Lee et al., 2025). Table 7 lists the complete set of hyperparameters used for all main experiments. Note that the latent rank r is not a free hyperparameter: it is deterministically

Table 7. **QAT hyperparameters** used for LittleBit-2 across all model configurations.

HYPERPARAMETER	VALUE
OPTIMIZER	ADAM
(β_1, β_2)	(0.9, 0.999)
WEIGHT DECAY	0.0
LR SCHEDULE	COSINE DECAY
WARMUP RATIO	2%
EPOCHS	5
SEQUENCE LENGTH	2048
INTERMEDIATE LOSS WEIGHT λ	10.0
LR	4E-5
JOINT-ITQ ITERATIONS T	50

computed from the target bits-per-parameter via the memory-budget inversion formula derived in Appendix H.